Tema 5
Persistencia

Mapeo objeto-relacional

Contenidos

1. Motivación.
2. Persistencia en Java con JDBC.
3. Persistencia en Java con ORM – Mapeo objeto-relacional.
4. Persistencia en Java con ORM – Gestión de entidades y acceso.

1. Motivación

El problema de guardar datos en lenguajes orientados a objetos

1. Motivación – Definición

La **persistencia** es la capacidad de almacenar y recuperar datos de una aplicación de forma que sobrevivan a la ejecución del programa.

RETOS

Fricción objeto–relacional.
Las aplicaciones se diseñan con objetos, pero los datos se almacenan en tablas.

Rendimiento.
El acceso a la BBDD es mucho más costoso que trabajar en memoria.

Escalabilidad y volumen de datos.
Algunas aplicaciones manejan grandes volúmenes de datos (*Big Data*).

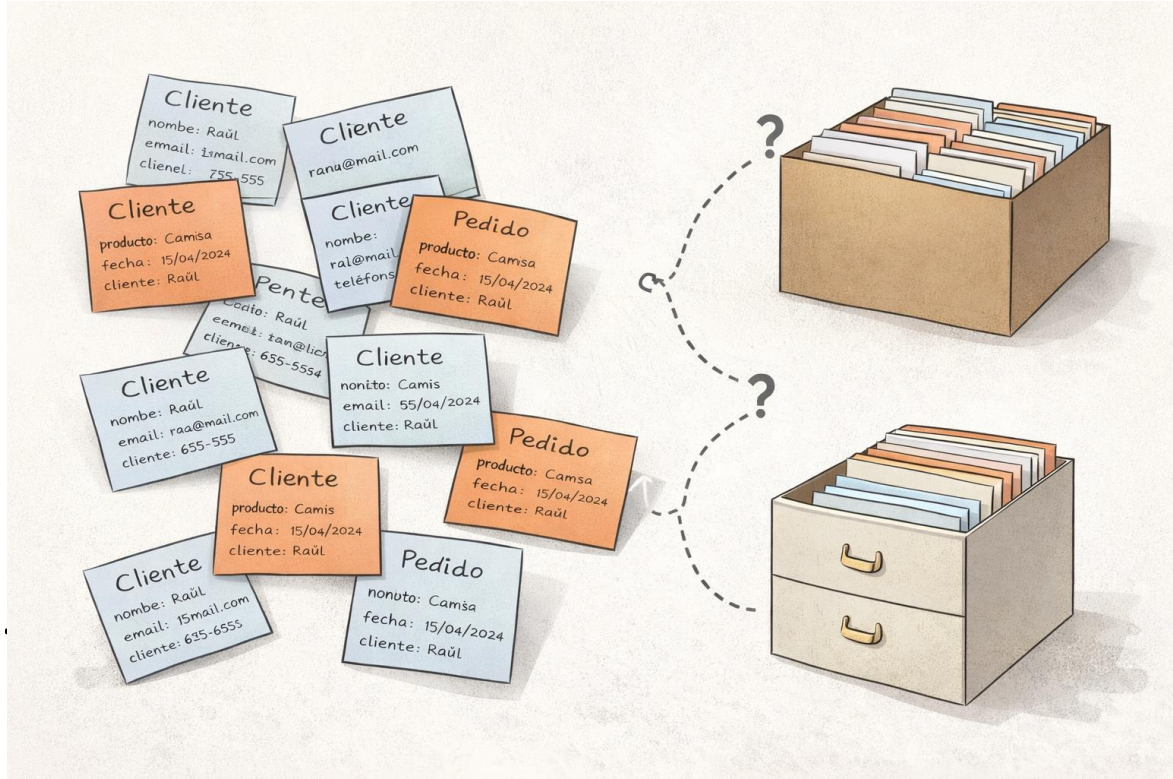
Gestión de concurrencia y consistencia.
Muchos usuarios pueden acceder y modificar los mismos datos.

Mantenibilidad del código.
Las consultas SQL pueden acoplar el código a la estructura de la base de datos.

1. Motivación – Persistencia de datos

¿Dónde guardamos los datos?

1. **Base de datos relacional.**
   - El tipo más habitual.
2. **Base de datos OO.**
   - Sería la opción ideal.
   - No extendidas ni maduras.
   - No han tenido el éxito que se esperaba.
3. **Bases de datos *NoSQL*.**
   - Grafos, documentos, columnas, etc.
   - No hay un estándar.
   - MongoDB, Cassandra, Neo4j, …



1. Motivación – Bases de datos OO

- Las bases de datos OO almacenan objetos completos, similares a los del programa.
- Modelo muy parecido al de la programación orientada a objetos

```
Cliente
  nombre: "Ana"
  email: "ana@mail.com"
  pedidos: [
      Pedido { producto: "Libro", fecha: "2024-05-10" }
      Pedido { producto: "Portátil", fecha: "2024-05-11" }
  ]
```

db4o

1. Motivación – Bases de datos NoSQL

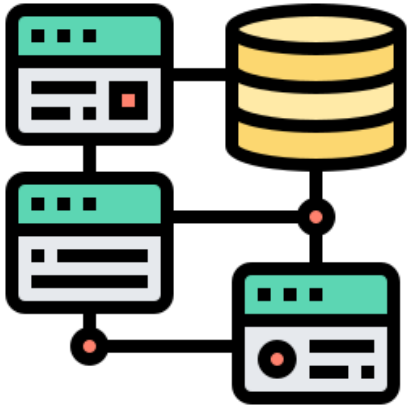
- Familia de BBDD que utilizan modelos de datos diferentes al relacional.
- Se diseñaron para problemas como:
  - Grandes volúmenes de datos.
  - Escalabilidad.
  - Flexibilidad en el esquema.

```
{
  "cliente": "Ana",
  "pedidos": [
    {"producto": "Libro"},
    {"producto": "Portátil"}
  ]
}
```

1. Motivación – Bases de datos relacionales

Las BBDDs relacionales se basan en unos principios simples.

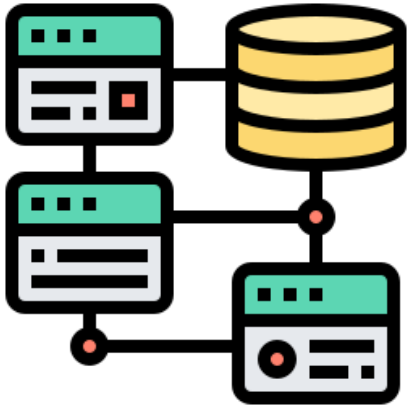
- **Información organizada en tablas.**
  - Los datos se almacenan en filas y columnas.
  - Cada celda contiene un único valor *(aunque algunos valores pueden ser nulos)*.
- **Identidad de los datos.**
  - Cada fila tiene una clave primaria.
  - La clave identifica de forma única cada registro.
- **Relaciones entre tablas.**
  - Las tablas se conectan mediante *claves ajenas*.
  - Permiten representar relaciones entre datos.



1. Motivación – Bases de datos relacionales

¿Por qué son tan populares las BBDD relacionales?

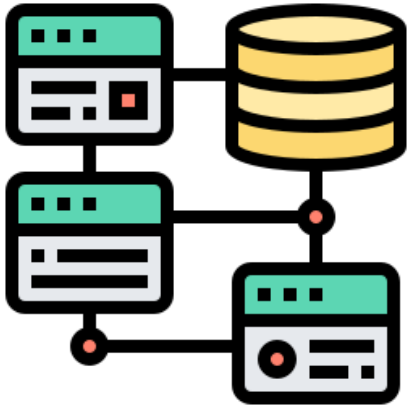
- **Las BBDDs relaciones son las más usadas en la industria.**
  Utilizadas en aplicaciones empresariales, sistemas web y sistemas transaccionales.
- **Tecnología madura y fiable.**
  Décadas de desarrollo, herramientas y buenas prácticas.
- **Estándar SQL.**
  Trabaja con diferentes motores *(PostgreSQL, MySQL, Oracle, etc.).*
- **Modelo relativamente simple.**
  Basado en tablas, filas y relaciones.
- **Fundamentos teóricos sólidos.**
  Basado en el modelo relacional *(álgebra relacional).*



1. Motivación – Bases de datos relacionales

Las BBDs relacionales tienen **limitaciones con respecto a OO**.

- **Limitada capacidad de modelado.**
  Sin mecanismos de herencia, polimorfismo, etc.
- **Complejidad de funciones básicas.**
  Por ejemplo, la instanciación de un único objeto complejo implica múltiples reuniones.
- **Diferencia semántica con OO.**
  Tipos limitados en BBDDs relacionales mientras que hay infinitas clases.



1. Motivación – Bases de datos relacionales

Cuenta

Numero (PK)	saldo	Cliente (FK)
200	85005	2689
505	40000	1003
821	50000	1210
426	35620	1003
005	29872	0300
…		

Cliente

Codigo (PK)	nombre	direccion	ciudad
11[illegible]	[illegible]a, A.	Gran Vía, 6	Murcia
03[illegible]0	López, B.	Ronda Norte, 3	Murcia
1003	Azorín, C.	Paseo Rosales, 9	Molina
2689	Pérez, D.	Plaza Mayor, 2	Patiño
...			

Vínculo Cliente-Cuenta

1. Motivación – Impedancia objeto-relacional

- **Mundos diferentes.**
  Las aplicaciones se diseñan utilizando objetos, mientras que las bases de datos relacionales utilizan tablas.
- **Impedancia objeto-relacional.**
  Diferencia conceptual entre el modelo orientado a objetos y el modelo relacional de las bases de datos.

Orientado a objetos

- Clases
- Objetos
- Referencias
- herencia

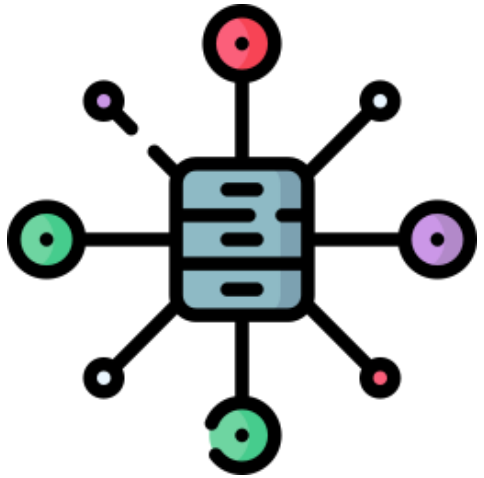
Relacional

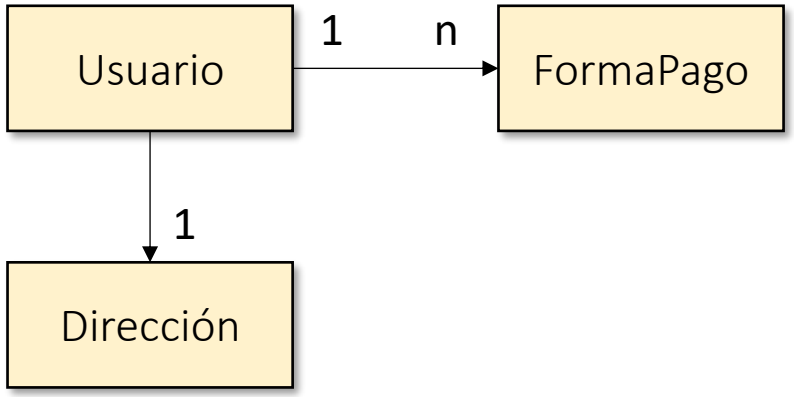
- Tablas
- Filas
- Claves primarias
- Claves ajenas

1. Motivación – Impedancia objeto-relacional

Problema 01
Granularidad de los tipos de datos.

- En OO se pueden crear nuevos tipos de datos fácilmente.
- No tenemos tipos de datos complejos en SQL.
- ¿Cómo mapeamos `Dirección`?

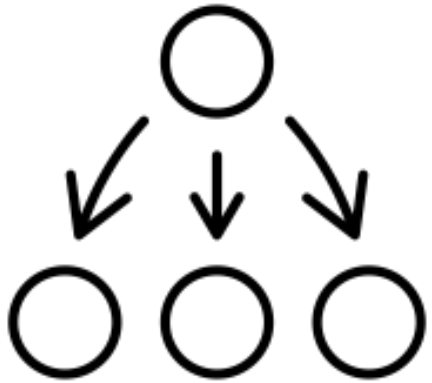


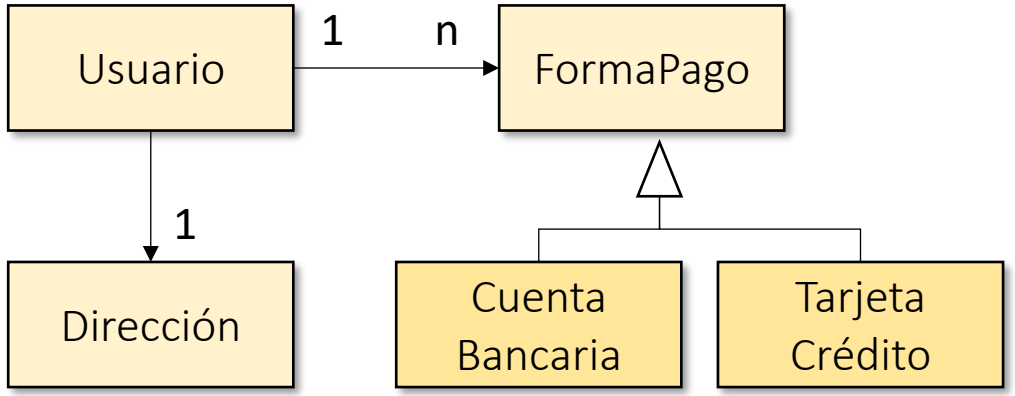


1. Motivación – Impedancia objeto-relacional

Problema 02
Herencia

- La herencia permite reutilizar comportamiento y tratar los objetos de forma polimórfica.
- Pero no hay herencia en el modelo relacional.
- Necesitamos **estrategias de mapeo**.





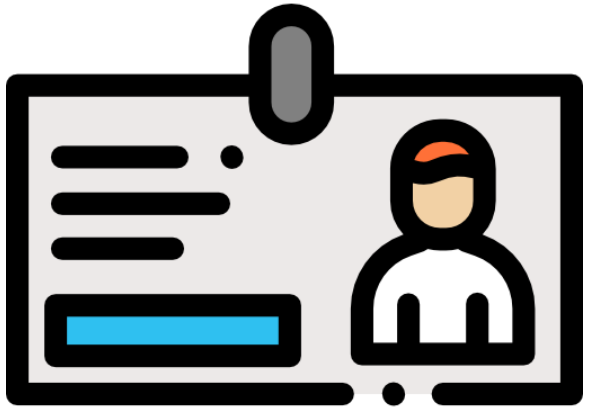
1. Motivación – Impedancia objeto-relacional

Problema 03
Identidad

En OO los objetos tienen identidad propia.

Dos objetos pueden ser:

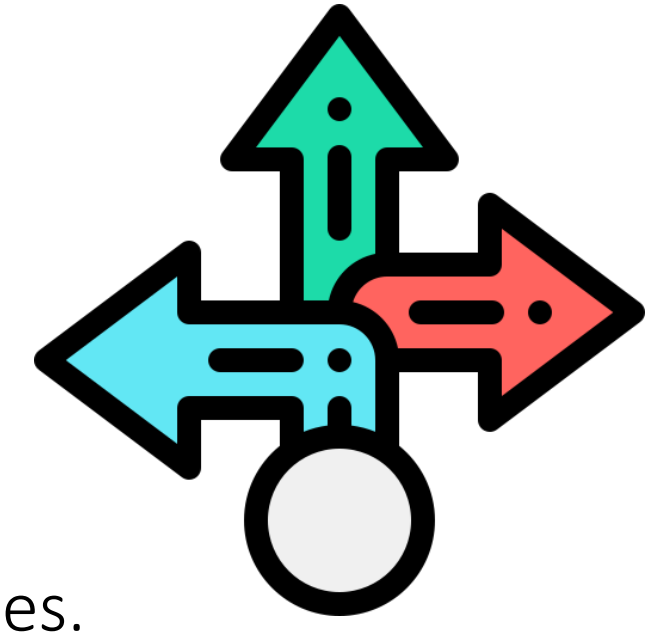
1. El mismo objeto en memoria.
2. Objetos distintos con los mismos datos.



En el **modelo relacional**, la identidad de un registro viene dada por su **clave primaria**.

1. Motivación – Impedancia objeto-relacional

Problema 04
Navegabilidad



En OO navegamos a través de referencias.

```
usuario.getFormasPago ()
  .stream ()
  .map (p -> p.getNumCuenta ())
```

En BBDD relacionales a través de consultas y reuniones.

```
SELECT fp.numCuenta
FROM Usuario u
JOIN FormaPago fp ON fp.usuario_id = u.id
WHERE u.id = 3;
```

1. Motivación – Impedancia objeto-relacional

El diseño OO no debe dirigir el diseño de la BBDD y viceversa.

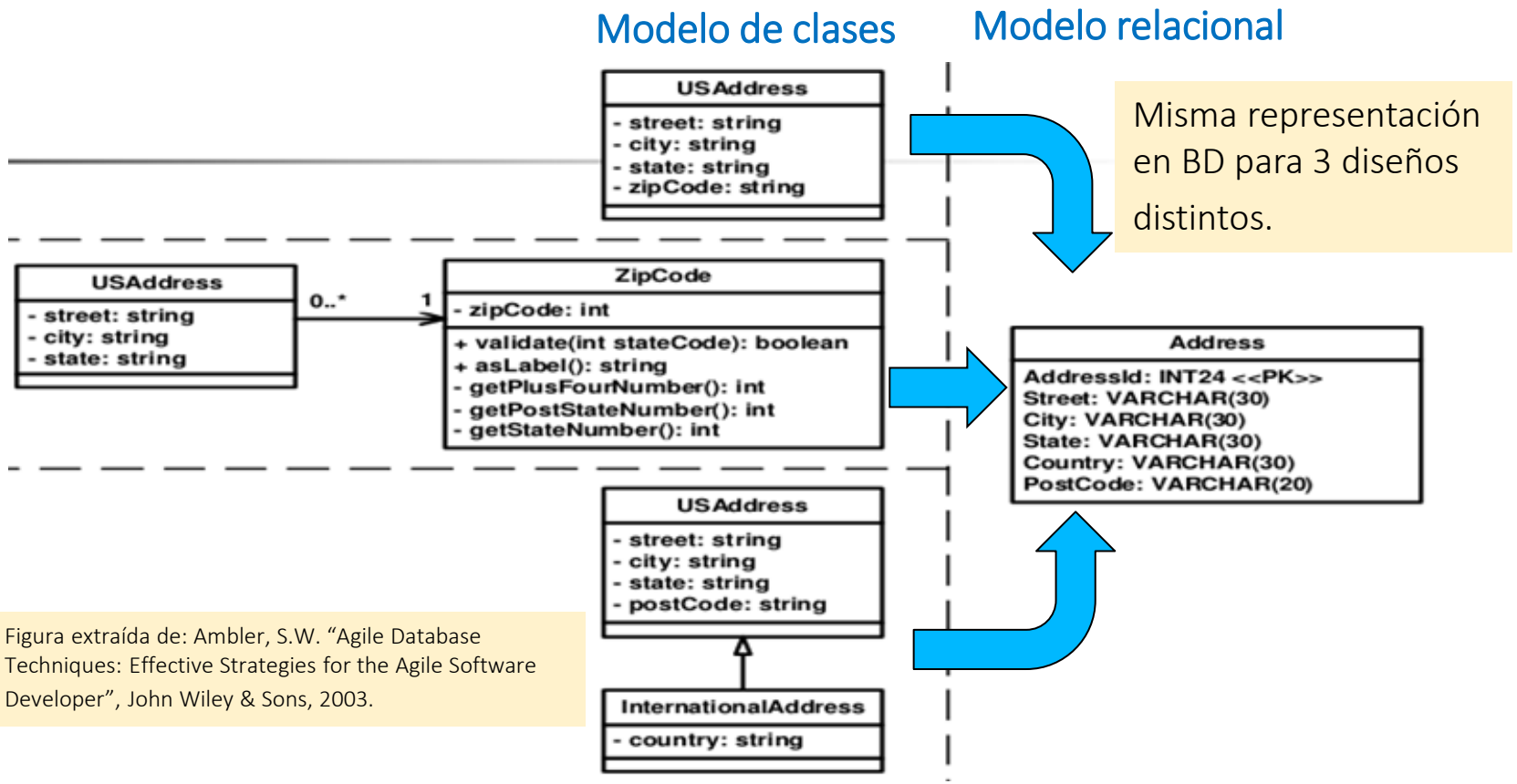


Figura extraída de: Ambler, S.W. "Agile Database Techniques: Effective Strategies for the Agile Software Developer", John Wiley & Sons, 2003.

- Cada modelo tiene su expresividad y herramientas.
- Considerar factores de rendimiento y escalabilidad.
- Diferentes opciones pueden llevar a representaciones independientes.

1. Motivación – Estrategias

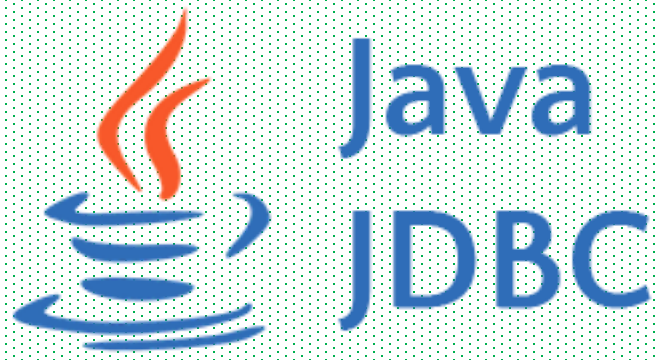
Para almacenar objetos en BBDD relacionales, existen dos enfoques principales:

- Persistencia manual.
- El programador escribe sus propias consultas.
- Control total del proceso.
- Mucho código *boilerplate*.

- Mapeo automático.
- Se trabaja principalmente con objetos.
- El framework genera SQL de manera interna.

2. Persistencia en Java con JDBC

¿Cómo persistir en JAVA con JDBC con control absoluto?



2. Persistencia con JDBC – Motivación

Escribir código ligado a una BBDD concreta trae problemas.

Acoplamiento

- El código queda ligado a un motor concreto *(MySQL, PostgreSQL, Oracle…).*
- Cambiar de BBDDs implica rehacer código.
- Dificulta la mantenibilidad del sistema.

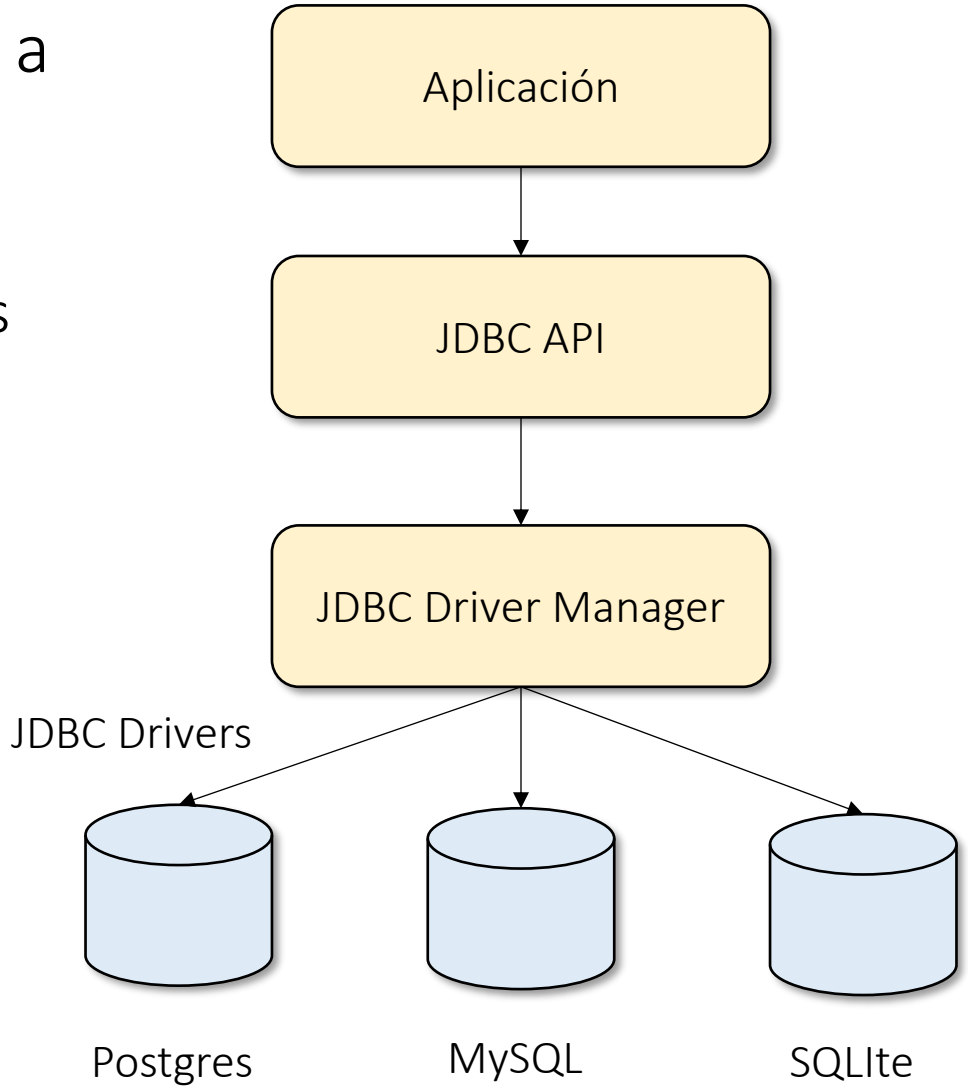
Portabilidad

- La aplicación no puede ejecutarse fácilmente en otros entornos.
- Las BBDDs ofrecen extensiones propias.
  Si no son estándar, tenemos menos portabilidad.

2. Persistencia con JDBC – Arquitectura

JDBC es la API estándar de Java para acceso a BBDD relacionales.

- **Capa de bajo nivel.**
  JDBC sólo proporciona interfaces y algunas clases básicas.
- **Cada BBDD proporciona su propio driver.**
  Los drivers proporcionan implementaciones concretas para cada motor de base de datos, lo que permite fácilmente pasar de una a otra.



2. Persistencia con JDBC – Clases principales

Clase	Descripción
`DriverManager`	Gestiona los drivers JDBC y establece conexiones con la BBDD.
`Connection`	Representa una sesión con una BBDD específica
`Statement`	Representa una consulta SQL estática (sin parámetros)
`PreparedStatement`	Representa una consulta SQL dinámica (con parámetros)
`CallableStatement`	Representa la invocación a un procedimiento almacenado
`ResultSet`	Representa el resultado de una consulta. Permite navegar las filas del resultado.
`SQLException`	Gestionar las excepciones de la BBDD.

2. Persistencia con JDBC – Configuración

- El driver tiene que estar accesible en `classpath`.
- En Maven:

¿Por qué?

```
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.49.1.0</version>
    <scope>runtime</scope>
</dependency>

<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.5</version>
    <scope>runtime</scope>
</dependency>
```

2. Persistencia con JDBC – Conexión

```java
public class BaseDeDatos implements AutoCloseable {
  private static final String URL = "jdbc:sqlite:basedatos.db";
  private Connection conexion;

  public BaseDeDatos() throws SQLException {
    conexion = DriverManager.getConnection(URL);
  }

  @Override
  public void close() throws Exception {
    try {
      if (conexion != null) {
        conexion.close();
      }
    } catch (SQLException e) {
      e.printStackTrace();
    }
  }
}
```

Para indicar a qué BBDD conectar, usamos una JDBC URL.
Especifica el driver específico y la ubicación del recurso.

Sesiones

- `getConnection` nos da una sesión activa con nuestra BBDD.
- Las conexiones consumen recursos, por lo que debemos de cerrarlas.
- Tienen estados
  (ACTIVA, CERRADA, EN TRANSACCIÓN, …)

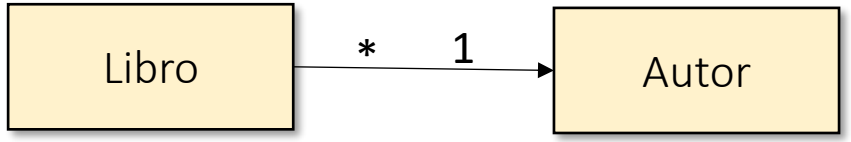
Excepciones

- Debemos de manejar excepciones, ya que estamos usando recursos externos.

2. Persistencia con JDBC – Crear tablas

```java
private void crearTablas() throws SQLException {
  String sqlAutores = """
    CREATE TABLE IF NOT EXISTS Autores (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      nombre TEXT NOT NULL,
    );""";

  String sqlLibros = """
    CREATE TABLE IF NOT EXISTS Libros (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      titulo TEXT NOT NULL,
      autor_id INTEGER,
      FOREIGN KEY (autor_id) REFERENCES Autores(id)
  );""";

  try (Statement stmt = conexion.createStatement()) {
    stmt.execute(sqlAutores);
    stmt.execute(sqlLibros);
  }
}
```

- El código SQL se construye como cadenas de texto.
- El programador debe conocer el modelo relacional.
- Las relaciones entre tablas las debemos de modelar nosotros mismos.
- JDBC permite enviar instrucciones SQL directamente a la base de datos mediante `statements`.



2. Persistencia con JDBC – Persistencia

```java
public void addLibro(Libro libro) {
  String sql = "INSERT INTO Libros (titulo, autor_id) VALUES (?, ?)";
  try (PreparedStatement pstmt = conexion.prepareStatement (
    sql,
    Statement.RETURN_GENERATED_KEYS
  )) {
    pstmt.setString(1, libro.getTitulo());
    pstmt.setLong(2, libro.getAutor().getId());
    pstmt.executeUpdate();
    ResultSet rs = pstmt.getGeneratedKeys();
    if (rs.next()) {
      long codigo = rs.getLong(1);
      libro.setId(codigo);
    } else {
      throw new IllegalStateException();
    }
  } catch (SQLException e) {
    e.printStackTrace();
    throw new RuntimeException(e);
  }
}
```

prepareStatement

- SQL parametrizado.
- Evita concatenar cadenas. Más seguro. Previene inyección de SQL.
- Tenemos placeholders posicionales (?) para insertar los datos.

`getGeneratedKeys`

JDBC permite recuperar claves generadas.

2. Persistencia con JDBC – Consultas

```java
public List<Autor> getAutores() {
  String consulta = "SELECT id, nombre FROM Autores";

  try (Statement stmt = conexion.createStatement();
    ResultSet rs = stmt.executeQuery(consulta)) {

    List<Autor> autores = new ArrayList<>();
    while (rs.next()) {
       Autor autor = new Autor(
         rs.getLong("id"),
         rs.getString("nombre")
       );
       autores.add(autor);
    }

    return autores;
  } catch (SQLException e) {
    e.printStackTrace();
    throw new RuntimeException(e);
  }
}
```

ResultSet

- funciona como un cursor: apunta a una fila del resultado y avanza con `next`
- Permite leer columnas de la fila actual.
- Los datos no se cargan todos en objetos automáticamente.

Mapeo manual objeto–relacional

- Construir la clase autor se tiene que hacer manualmente.

Implicaciones de memoria

- Fijaros que aquí se cargan todos los autores en una lista.
- Serían buenas prácticas aplicar paginación.

2. Persistencia con JDBC – Ventajas y desventajas

Ventajas

- Sencillo de usar.
- Control sobre cómo se realiza la persistencia.
- Ejecución más rápida de consultas simples (no siempre hace falta el mapeo).

Desventajas

- Creación de gran cantidad de código manual.
- Riesgo de errores.
- No hay gestión automática de transacciones.
- No hay *lazy loading.*

3. Persistencia en Java con ORM

Mapeo objeto-relacional.

3. Persistencia con ORM

Cuando usamos JDBC:

- Escribimos SQL manualmente.
- Convertimos filas en objetos a mano.
- Mezclamos lógica de negocio con persistencia.
- Repetimos mucho código similar (*boiler plate*).

Además del problema de fondo:

JDBC trabaja con tablas, pero nuestras aplicaciones trabajan con objetos.

3. Persistencia con ORM – Mapeo objeto-relacional

El **mapeo objeto-relacional (ORM)** es la persistencia automatizada y transparente de objetos en las tablas de un sistema de gestión de BBDD relacional (RDBMS), utilizando metadatos que describen la relación entre las clases de la aplicación y el esquema de la BBDD SQL.

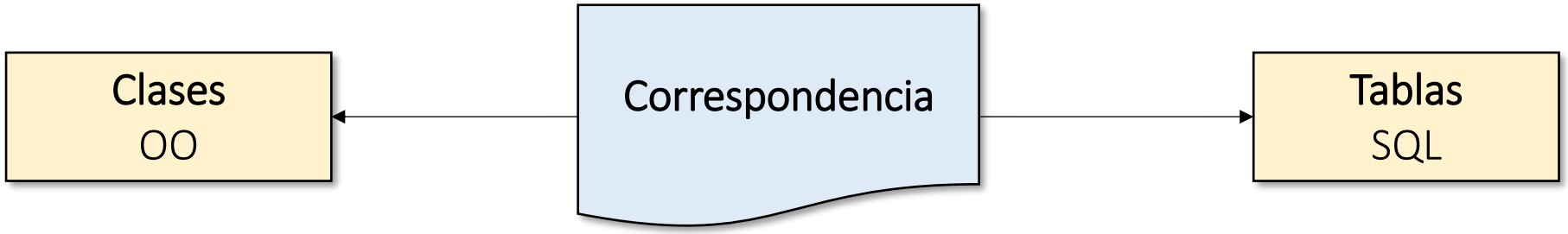
- ORM es una técnica para traducir automáticamente entre objetos y tablas *(y viceversa)*.
- Evita la necesidad de escribir consultas SQL manualmente.
- ORM mejora el nivel de abstracción.

Mundo OO	Mundo relacional
Clase	Tabla
Objeto	Fila
Atributo	Columna
Referencia	Clave ajena

3. Persistencia con ORM – Soluciones disponibles

El mapeo objeto–relacional es una técnica común en muchos lenguajes.

Y todos trabajan de manera bastante similar.



3. Persistencia con ORM – ¿Qué mapeamos?

- Entidades.
- Asociaciones.
  - Uno a uno.
  - Uno a muchos.
  - Muchos a muchos.
- Herencia (distintas estrategias).
  - Una tabla por jerarquía.
  - Una tabla por clase (tanto padre como hijos)
  - Una tabla por subclase.
- Otros elementos.
  - Colecciones
  - Clases asociativas.

Modelo OO	Modelo relacional
Clase	Tabla
Objeto	Fila
Propiedad	Columna
OID *	Clave primaria
Visibilidad de atributo	Clave ajena

3. Persistencia con ORM – JPA

Java Persistence API (JPA) es la especificación Java acerca de cómo persistir objetos en BBDDs relacionales.

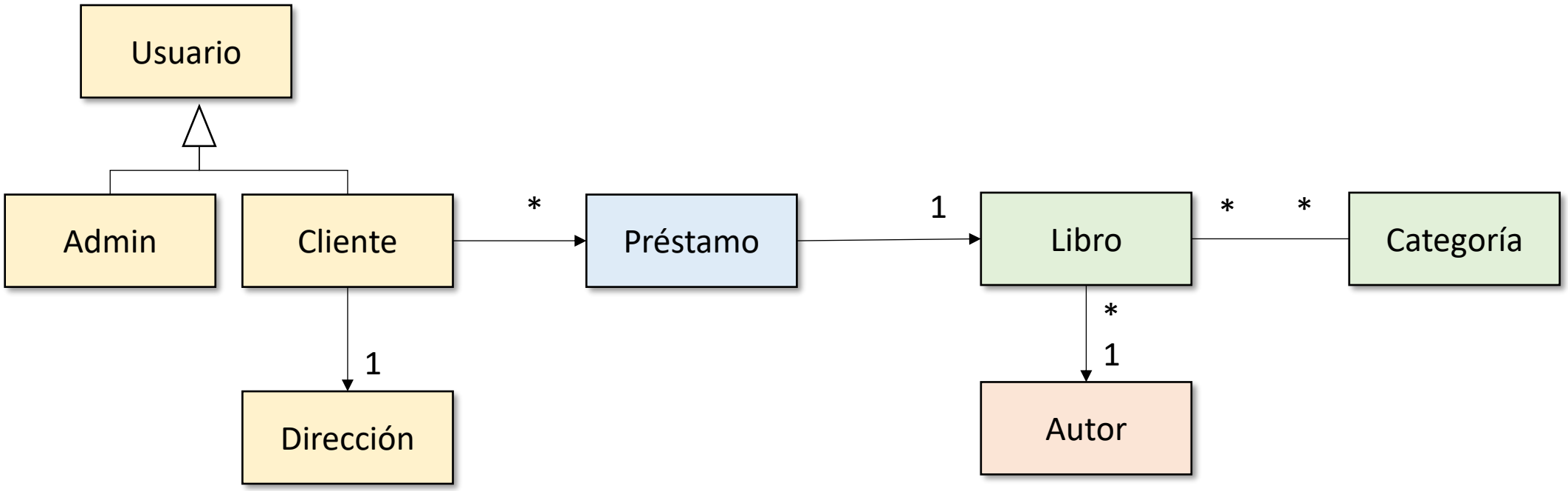
- Estandariza la estructura del mapeo objeto-relacional en Java
- Diferentes proveedores: Hibernate, EclipseLink, etc.

Terminología básica

- Entity.
- EntityManager.
- Persistence Context.
- Criteria API.
- Persistence Unit.
- JPQL (*Java Persistence Query Language*).

3. Persistencia con ORM – Caso de estudio

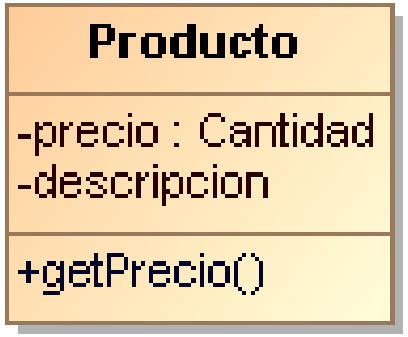
Vamos a ver **distintas técnicas de mapeo** con un ejemplo de una librería.
Relaciones uno a uno, muchos a muchos, uno a muchos, herencia, tipos de datos, …



3. Persistencia con ORM – Entidades

Estrategia para mapear entidades.

- Crear una tabla por cada clase persistente.
- Crear una columna por cada atributo.
- Usar el mismo nombre del atributo.
- Añadir una columna identificadora (ID)




```
CREATE TABLE Producto (
  id INTEGER PRIMARY KEY,
  precio NUMBER(10, 2),
  descripción TEXT
)
```

Dificultades.

- Los tipos de datos pueden no coincidir.
- Situaciones complejas donde un objeto puede mapearse en varias tablas *(o varias clases en la misma tabla).*
- El identificador de la BBDD no suele ser algo del dominio.

3. Persistencia con ORM – Entidades

JPA funciona mediante anotaciones.

`@Entity`

- Indica que una clase debe ser persistida.
- Obligatorio asociar anotación `@Id` para indicar cuál es la clave primaria.

`@Table`

- Indicar propiedades de la tabla, como el nombre de la tabla *(Opcional)*.

```
@Entity
@Table(name = "Autores")
public class Autor {
  @Id
  private String nombre;
}
```

¿Es esta realmente la clave primaria para un autor?

3. Persistencia con ORM – Entidades – Identificadores

En ORM, cada objeto persistente necesita una clave primaria.

Claves subrogadas.

- No tienen significado en el dominio.
- Generados automáticamente.
- Son estables, aunque cambien los datos del objeto.

```
@GeneratedValue(GenerationType.AUTO)
```

- Pide usar una estrategia para generar automáticamente las claves.
- La estrategia concreta depende de la base de datos.

3. Persistencia con ORM – Entidades – Identificadores

Distintas estrategias:

1. **Identidad** (`GenerationType.IDENTITY`)
   - Usa columnas auto-incrementales.
   - Ampliamente soportado.
   - *Batch updates* ya no son posibles.
2. **Secuencia** (`GenerationType.SEQUENCE`)
   - Usa **secuencias** de bases de datos.
   - Permite obtener el ID antes de insertar.
   - Eficiente en operaciones masivas.
3. **Tabla** (`GenerationType.TABLE`)
   - Simula una secuencia. Más portable, menos eficiente.

```
@Entity
@Table(name = “Autores”)
public class Autor {
  @Id
  @GeneratedValue(strategy=GenerationType.SEQUENCE)
  private Long id;
}
```

3. Persistencia con ORM – Entidades – Identificadores

La diferencia clave entre estrategias está en cuándo se obtiene el ID.
Esto afecta al rendimiento en inserciones masivas.

Generación ANTES del INSERT (`SEQUENCE`)

1. El ORM solicita un nuevo identificador.
2. El objeto recibe su ID.
3. Se ejecuta el INSERT usando ese ID.

Generación DESPUÉS del INSERT (`IDENTITY`)

1. Se ejecuta el INSERT.
2. La base de datos genera el ID automáticamente.
3. El ORM recupera el identificador generado.

Tudose, C., Bauer, C., & King, G. (2023). *Java Persistence with Spring Data and Hibernate*. Simon and Schuster.

3. Persistencia con ORM – Entidades – Datos

- El lenguaje de programación ofrece tipos que no siempre existen en BBDD.
- Algunos tipos de datos pueden mapearse de distintas maneras:
  Por ejemplo, un booleano puede ser un `tinyint(1)`.
- Algunos tipos requieren mapeos especiales, como datos binarios o datos muy grandes.

3. Persistencia con ORM – Entidades – Datos

Name	Java type	ANSI SQL type
Integer	`int, java.lang.Integer`	`INTEGER`
Long	`long, java.lang.Long`	`BIGINT`
Short	`short, java.lang.Short`	`SMALLINT`
Float	`float, java.lang.Float`	`FLOAT`
Double	`double, java.lang.Double`	`DOUBLE`
Byte	`byte, java.lang.Byte`	`TINYINT`
Boolean	`boolean, java.lang.Boolean`	`BOOLEAN`
Big_decimal	`java.math.BigDecimal`	`NUMERIC`
Big_integer	`java.math.BigInteger`	`NUMERIC`

3. Persistencia con ORM – Entidades – Datos

@Lob indica que el atributo puede ser muy grande.

El ORM decide el mapeo:

- `byte[] a BLOB`
  datos binarios: imágenes, ficheros
- `String a CLOB`
  texto muy largo

Consideraciones de diseño

- Guardar binarios en BBDD afecta al rendimiento.
- **IDEA:** Almacenar el fichero y guardar solo la ruta.

```
@Entity
@Table(name = "clientes")
public class Cliente extends Usuario {
  @Lob
  private byte[] fotografia;

  @Lob
  private String biografia;
}
```

Mapeado a un BLOB

Mapeado a un CLOB

3. Persistencia con ORM – Entidades – Value Types

Hay objetos contenidos en otros que:

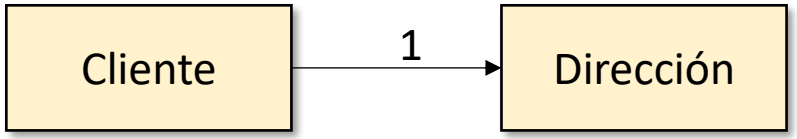
- No tienen identidad propia.
- No existen por sí mismos.
- Formen parte del objeto principal.

Solución ORM

Aplanar el objeto, guardando sus atributos como columnas de la tabla principal

`@Embeddable`
Tipo reutilizable

`@Embedded`
Se integra en la entidad.




```
@Entity
@Table(name = "Clientes")
public class Cliente extends Usuario {
  private Direccion direccion;
}
```

```
@Entity
@Embeddable
public class Direccion {
  private String calle;
  private int numero;
}
```

3. Persistencia con ORM – Asociaciones

Los objetos no están aislados, se relacionan:

- Un objeto puede contener a otro.
- Un objeto puede referenciar a muchos.
- Las relaciones pueden ser bidireccionales.
- Existen relaciones de herencia.

Problema

- Las BBDDs no almacenan objetos conectados, sino almacenan tablas relacionadas mediante claves foráneas.
- El ORM debe traducir relaciones de objetos en relaciones entre tablas.

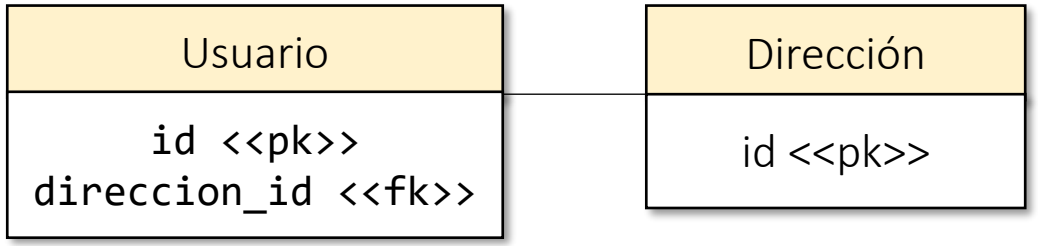
3. Persistencia con ORM – Asociaciones – Uno a uno

Opciones

1. Mezclar en una tabla (ya lo vimos con Value Types)
2. Si los conceptos son importantes se mantienen *(se crea una FK)*

Detalles

- La relación puede ser unidireccional o bidireccional. *El esquema de BBDD no cambia.*
- Si la relación es obligatoria el ORM genera una restricción `NOT NULL`.




```
@Entity
public class Usuario {
  @Id
  Long id;
  @OneToOne
  @JoinColumn(unique=true)
  Direccion direccion;
}

@Entity
public class Direccion {
  @Id
  Long id;
  @OneToOne(mappedBy="direccion")
  Usuario usuario;
}
```

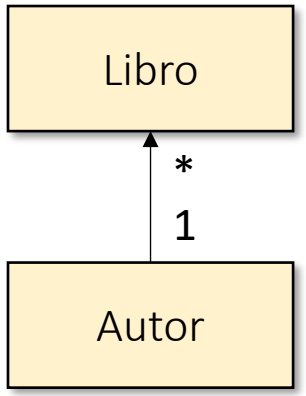
En este ejemplo **Dirección** no es *embeddable*

3. Persistencia con ORM – Asociaciones – Uno a muchos

- Una entidad se relaciona con varias del otro tipo.
- La clave ajena se guarda en el lado "*muchos*".

Ejemplo:

Un autor puede escribir varios libros.




```
@Entity
Class Libro {
  @Id
  Long id;

  @ManyToOne
  @JoinColumn(name="autor_id", nullable=false)
  Autor autor;
}
```

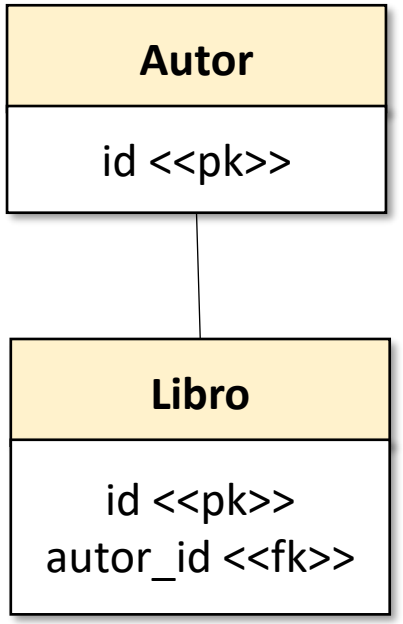
Un libro está escrito por un autor. El mismo autor puede escribir varios libros.

La FK va en el lado de "muchos"

3. Persistencia con ORM – Asociaciones – Uno a muchos

- Si queremos hacer la relación **uno a muchos bidireccional**, no tenemos que cambiar la BBDD.
  La relación es la misma.
- Podemos añadir en la clase "Uno" cómo se mapea `(mappedBy)`.

```
@Entity
Class Autor {
  @Id
  Long id;

  @OneToMany(mappedBy="autor")
  List<Libro> libros;
}
```



3. Persistencia con ORM – Asociaciones – Muchos a muchos

Cuando muchas entidades se relacionan con muchas entidades.

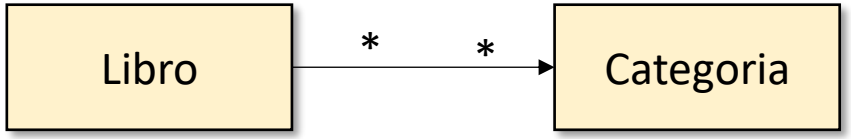
Ejemplo:

- *Un libro puede pertenecer a varias categorías.*
- *Una categoría contiene muchos libros.*

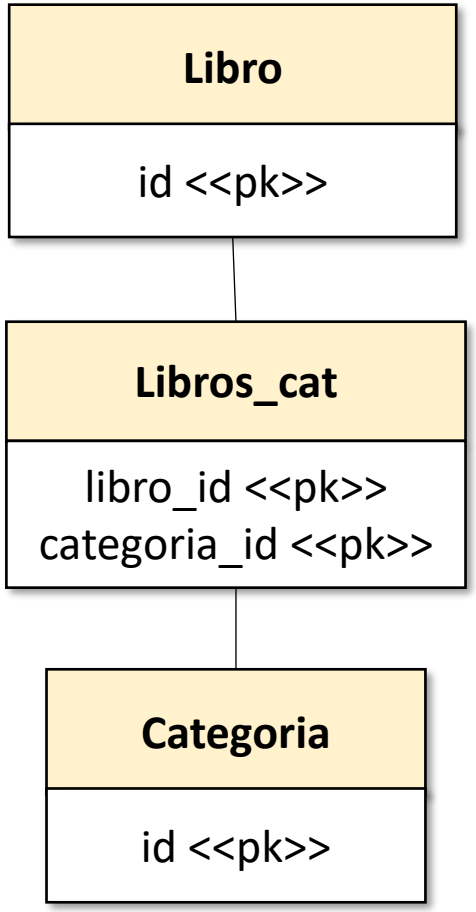
En BBDD relacionales la relaciones muchos a muchos se simulan así:

- Una tabla intermedia.
- Con dos claves ajenas.
- Que forman la clave primaria compuesta.

```
@Entity
public class Libro {
  @Id
  Long id;
  @ManyToMany
  @JoinTable(name = “drama")
  List<Categoria> categorias;
}

@Entity
public class Categoria {
  @Id
  Long id;
  @ManyToMany(mappedBy=“categorias”)
  List<Libros> libros;
}
```



3. Persistencia con ORM – Asociaciones – Muchos a muchos

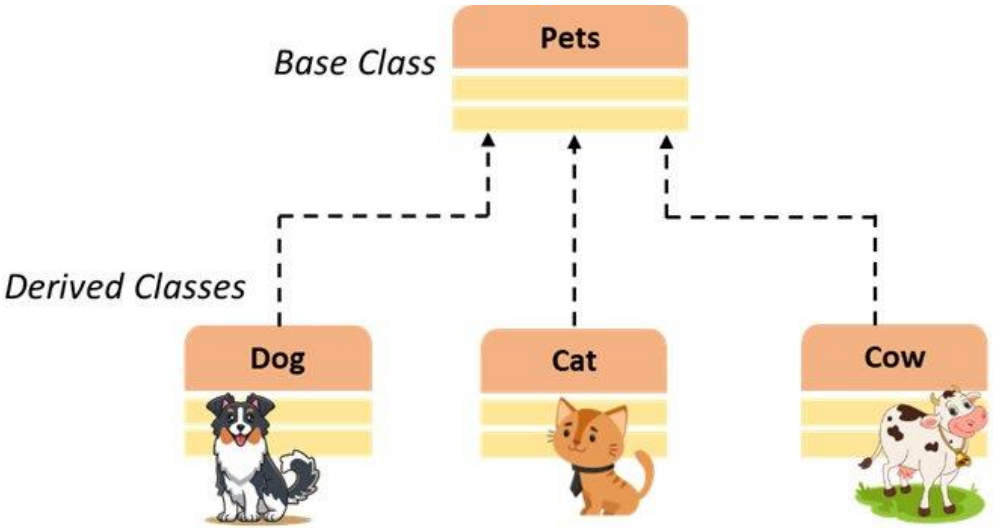



```
@Entity
public class Libro {
  @Id
  Long id;
  @ManyToMany
  @JoinTable(name = “drama")
  List<Categoria> categorias;
}

@Entity
public class Categoria {
  @Id
  Long id;
  @ManyToMany(mappedBy=“categorias”)
  List<Libros> libros;
}
```

3. Persistencia con ORM – Herencia

Las bases de datos relacionales no soportan herencia directamente.

El ORM debe traducir jerarquías de clases a tablas.

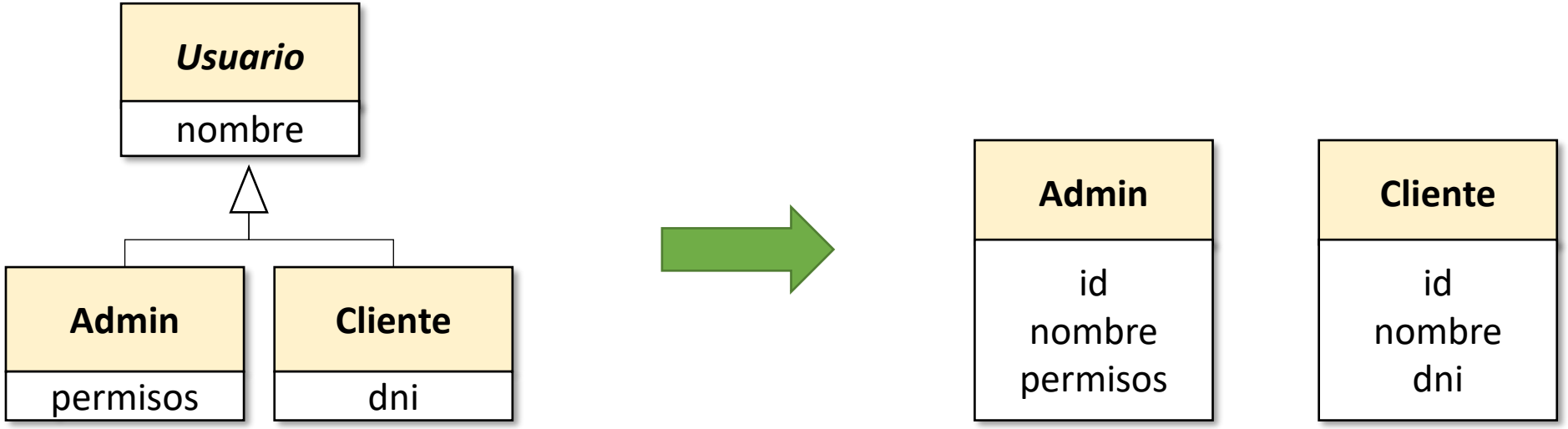


¿Cómo lo guardo en tablas?

3. Persistencia con ORM – Herencia

En JPA el mapeo por defecto es usar **clases separadas**.

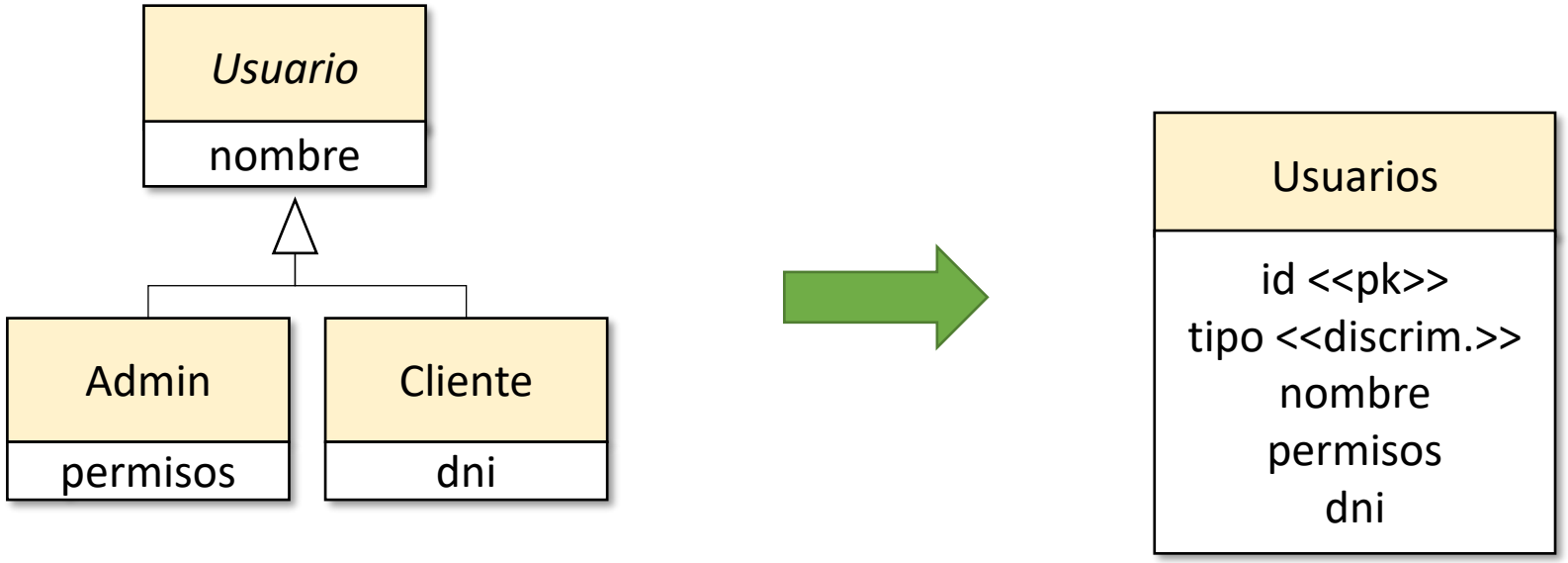
- Por defecto JPA no mapea los atributos de la superclase.
- Añadir la anotación `@MappedSuperclass` a la clase padre.
- Cada tabla contendrá los atributos de las superclases.
- No permite consultas polimórficas.



3. Persistencia con ORM – Herencia

Estrategia 1: Tabla única para toda la jerarquía (SINGLE TABLE)

- Una tabla con columna tipo, que se usa como discriminante.
- Atributos de subclases opcionales.



3. Persistencia con ORM – Herencia

Estrategia 1: Tabla para toda la jerarquía (SINGLE TABLE)

```
@Entity
@Table(name = “Usuarios”)
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "tipo_usuario",
                     discriminatorType = DiscriminatorType.STRING)
public abstract class Usuario {

}

@Entity
@Table(name = “Clientes”)
@DiscriminatorValue("CLIENTE")
public class Cliente extends Usuario {

}
```

3. Persistencia con ORM – Herencia

Estrategia 1: Tabla para toda la jerarquía (SINGLE TABLE)

Ventajas

- No hacen falta reuniones para recuperar una clase *(consultas rápidas).*

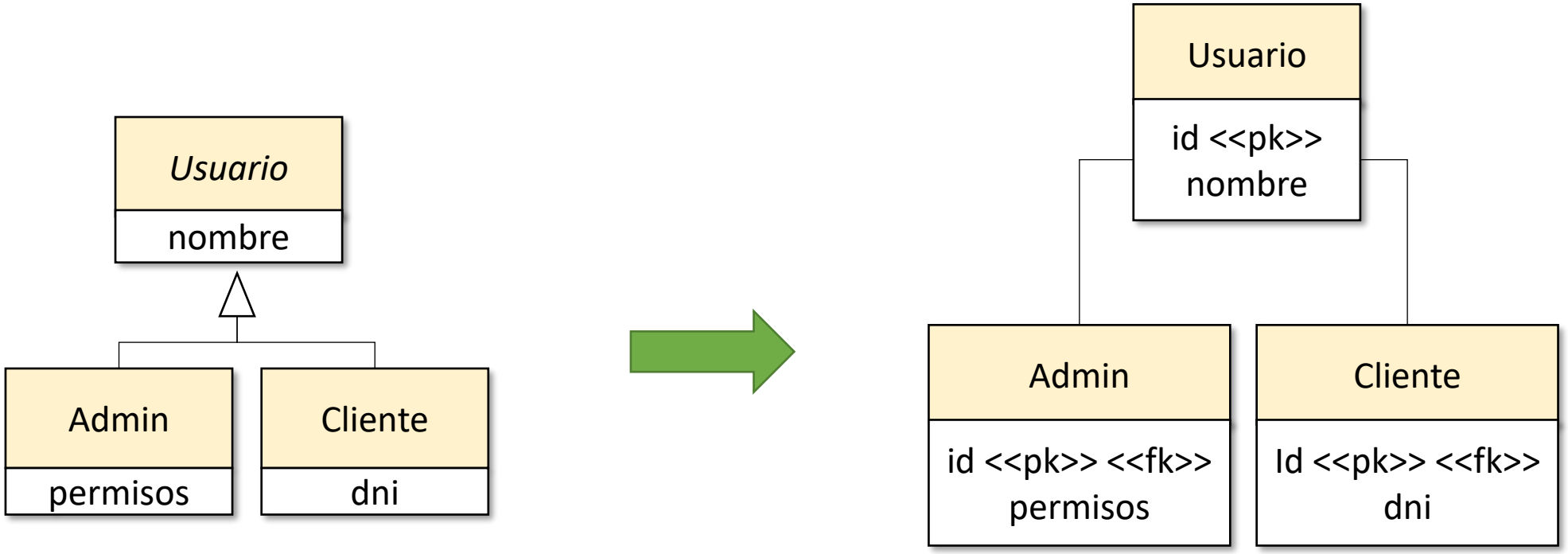
Inconvenientes

- Nulos en el almacenamiento.
- Tabla grande y poco limpia.
- Mantener la consistencia de los atributos con su clase
- Difícil gestionar relaciones con subclases

3. Persistencia con ORM – Herencia

Estrategia 2: Tabla por clase (JOINED)

- Una tabla por clase, pero los datos no se aplanan en las subclases
  - Se hace un *join* para recuperar los datos de cada clase



3. Persistencia con ORM – Herencia

Estrategia 2: Tabla por clase (JOINED)

- Una tabla padre y tabla para cada subclase.

Ventajas

- No hay nulos.
- Modelo normalizado.
- Es fácil extender la herencia.
- Bueno para el almacenamiento.

Inconvenientes

- Peor rendimiento por los JOINs.

```
@Entity
@Table(name = "Usuarios")
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Usuario {
  @Id
  @Inheritance(strategy=GenerationType.SEQUENCE)
  private Long id;
}
```

```
@Entity
@Table(name = "Clientes")
public class Cliente extends Usuario {
}
```

3. Persistencia con ORM – Herencia

Estrategia 3: Tabla por subclase (TABLE_PER_CLASS)

- Cada subclase tiene tabla completa.
- No hay una tabla para la superclase.

Ventajas

- No hay JOINs para subclases.
- Consultas rápidas por tipo.

Inconvenientes

- Datos duplicados.
- UNION para consultas polimórficas.
- No permite representar generalizaciones incompletas.

```
@Entity
@Table(name = "Usuarios")
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class Usuario {
  @Id
  @Inheritance(strategy=GenerationType.SEQUENCE)
  private Long id;
}
```

3. Persistencia con ORM – Herencia

Resumen de las distintas estrategias para mapear la herencia.

Estrategia	Idea	Ventaja	Problema
Tabla única	Una tabla para toda la jerarquía.	Consultas simples.	Muchos NULL.
Tabla por clase padre-hija	Cada clase tiene su tabla.	Modelo limpio.	Requiere JOIN.
Tabla por subclase	Solo tablas finales.	Consultas rápidas por tipo.	Datos duplicados.

3. Persistencia con ORM – Herencia

¿Qué estrategia escogemos?

TABLE_PER_CLASS
(por subclase)

- Cuando NO necesitamos polimorfismo.
- Consultas simples por tipo concreto.
- Subclases muy diferentes.

SINGLE_TABLE
(una única tabla)

- Cuando hay consultas polimórficas frecuentes.
- Subclases parecidas en datos.
- Se prioriza rendimiento de lectura.

JOINED
(tabla por clase)

- Cuando hay muchas subclases distintas
- Se necesita polimorfismo
- Modelo más normalizado

3. Persistencia con ORM – Otros

Colecciones de valores

Se pueden tratar como relaciones **uno a muchos**, con una nueva tabla.

```
// EJEMPLO: Un Usuario con varios teléfonos
@Entity
class Usuario {
  @ElementCollection
  List<String> telefonos;
}
```

Clases asociativas o atributos en una relación.

Se tratan como una muchos a muchos; es decir, con una tabla intermedia que contiene más atributos.

```
// EJEMPLO
// Matrícula de alumno en curso con nota
@Entity
class Matricula {
  @ManyToOne Alumno alumno;
  @ManyToOne Curso curso;
  double nota;
}
```

4. Persistencia en Java con ORM

Gestión de entidades y control de acceso.

4. Persistencia con ORM – Motivación

Hasta ahora hemos visto:

- Cómo mapear clases y tablas.
- Cómo representar relaciones.
- Cómo persistir estructuras complejas.

Ahora…

- ¿Quién gestiona los objetos mientras la aplicación está en ejecución?
- ¿Cuándo se sincronizan los cambios con la base de datos?
- ¿Cómo se controla el ciclo de vida de una entidad?

4. Persistencia con ORM – Motivación

Un posible enfoque para manejar la persistencia es **Active Record.**

- El propio objeto guarda y recupera sus datos.
- Mezcla lógica de negocio y persistencia.
- Más simple, pero menos flexible.

Ejemplo:

```
usuario.save()
pedido.delete()
```

4. Persistencia con ORM – Motivación

Otro enfoque es **Data Mapper (el que usamos aquí).**

- La persistencia se delega a un componente externo.
- Separación entre dominio y base de datos.
- Más complejo, pero más mantenible y escalable.

Ejemplo:

```
entityManager.persist(usuario)
entityManager.remove(pedido)
```

4. Persistencia con ORM – Contexto de persistencia

Un **contexto de persistencia** es el conjunto de entidades gestionadas en memoria durante una unidad de trabajo.

- Se puede ver como una caché de primer nivel.
- Garantiza identidad (una fila = un objeto).
- Detecta cambios automáticamente.
- Sincroniza con la BBDD.

https://docs.oracle.com/javaee/7/api/javax/persistence/EntityManager.html

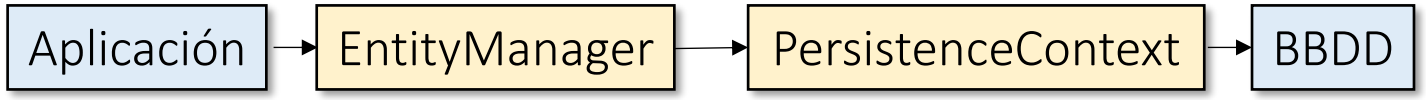
4. Persistencia con ORM – Entity Manager

El **EntityManager** se utiliza para crear y eliminar instancias de entidades persistentes, para encontrar entidades por su clave principal y para realizar consultas sobre entidades.

Es el **gestor del Contexto de Persistencia**.

Permite:

- Persistir entidades.
- Eliminarlas.
- Buscarlas.
- Sincronizar cambios.
- Controlar transacciones.



https://docs.oracle.com/javaee/7/api/javax/persistence/EntityManager.html

4. Persistencia con ORM – Ciclo de vida

El **EntityManager** gestiona entidades en una **unidad de trabajo**.
(Desde que se crea hasta que se cierra)

¿Qué hace?

- Mantiene entidades en memoria.
- Detecta cambios automáticamente.
- Sincroniza con la BBDD al hacer commit.

¿Dónde se crea?

- Aplicaciones web: Crear uno por petición
- Aplicaciones de escritorio: Crear al iniciar.

```
// Comenzamos a trabajar
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();

// Crear entidades
Autor autor = new Autor("Isaac Asimov");
Libro libro = new Libro("Fundación");

// Asociar entidades
libro.setAutor(autor);

// Persistir (guardar en BD)
em.persist(autor);
em.persist(libro);
em.getTransaction().commit();
em.close();
```

4. Persistencia con ORM – Gestionar objetos

```
em.getTransaction().begin();

// Obtener una entidad por su ID
Autor autor = em.find(Autor.class, 42L);

// La entidad queda gestionada (managed)
autor.setNombre("Jordi Sierra i Fabra");

// Recuperar de nuevo la misma entidad
Autor autor2 = em.find(Autor.class, 42L);

// Es la MISMA instancia en memoria
System.out.println (autor == autor2);
em.getTransaction().commit();

// Aquí se sincroniza con la BD
em.getTransaction().begin();
```

Buscar un objeto por ID

Gestionado por el EntityManager, en estado *Persisted*

JPA detecta automáticamente cambios *(dirty checking)*

Se inserta realmente al hacer commit

4. Persistencia con ORM – Persistencia de relaciones sin cascada

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();

// Creamos y relacionamos las instancias
Autor autor = new Autor("William Goldman");
Libro l1 = new Libro("La princesa prometida", autor);
Libro l2 = new Libro("Marathon Man", autor);
autor.addLibro(l1);
autor.addLibro(l2);

// Hay que guardar TODO manualmente
em.persist(autor);
em.persist(l1);
em.persist(l2);

em.getTransaction().commit();
```

- JPA no guarda automáticamente los objetos relacionados.
- El EntityManager solo persiste lo que le dices explícitamente

4. Persistencia con ORM – Persistencia de relaciones con cascada

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();

// Creamos y relacionamos las instancias
Autor autor = new Autor("William Goldman");
Libro l1 = new Libro("La princesa prometida", autor);
Libro l2 = new Libro("Marathon Man", autor);

autor.addLibro(l1);
autor.addLibro(l2);

// Sólo guardamos el autor
em.persist(autor);

em.getTransaction().commit();
```

Sin embargo, esto se puede simplificar usando una cascada.

Para eliminar también en cascada

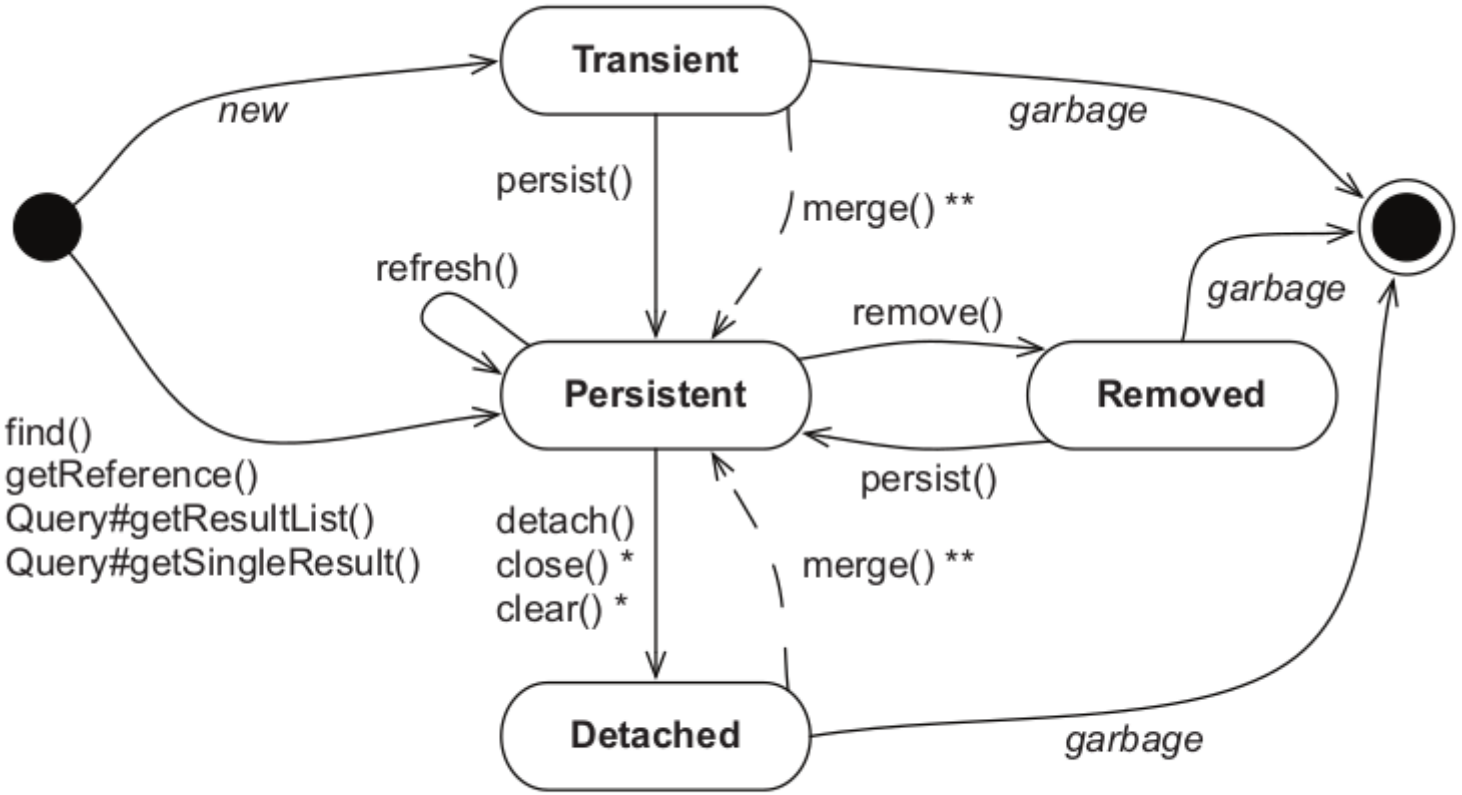
```
@OneToMany (
  mappedBy="autor",
  cascade = {
    CascadeType.PERSIST,
    CascadeType.REMOVE
  }
)

private List<Libro> libros;
```

4. Persistencia con ORM – Eliminar objetos

```
em.getTransaction().begin();

// Buscamos al autor
Autor autor = em.find(Autor.class, 42L);

// pasa a estado REMOVED
em.remove(autor);

// Se borra definitivamente en la BBDD
em.getTransaction().commit();
```

- Al eliminar entidades `(remove)` no se borra en la BBDD al instante, sino que la marca para borrar.
- La BBDD se actualiza al confirmar la transacción.

4. Persistencia con ORM – Estados



* Affects all instances in the persistence context
** Merging returns a persistent instance, original doesn't change state

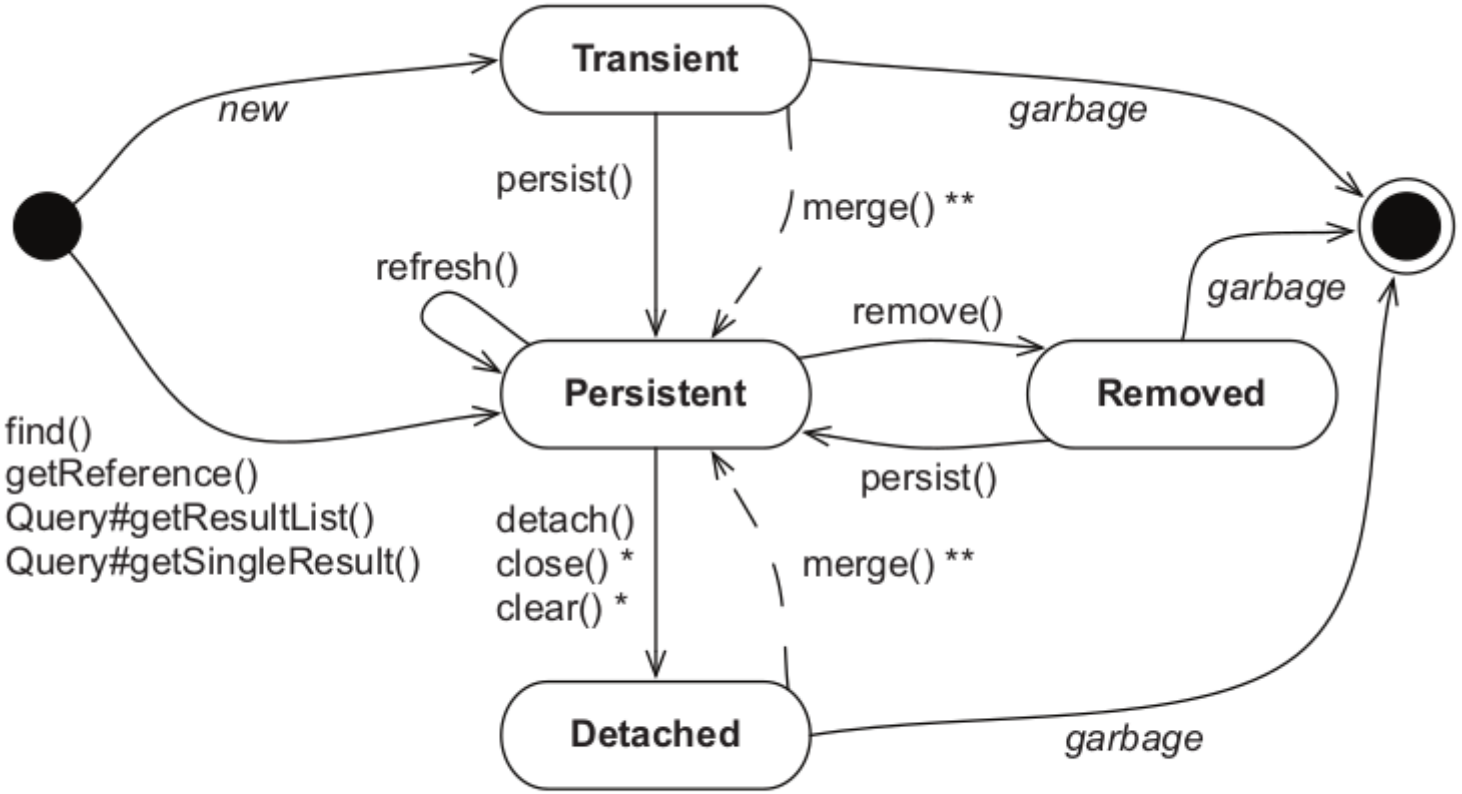
Transient

- Objetos recién creados con **new**.
- Solo existe en memoria.
- No está en la BBDD.
- El EntityManager no lo controla

Puede pasar a:

- Persistent → `persist()`

4. Persistencia con ORM – Estados



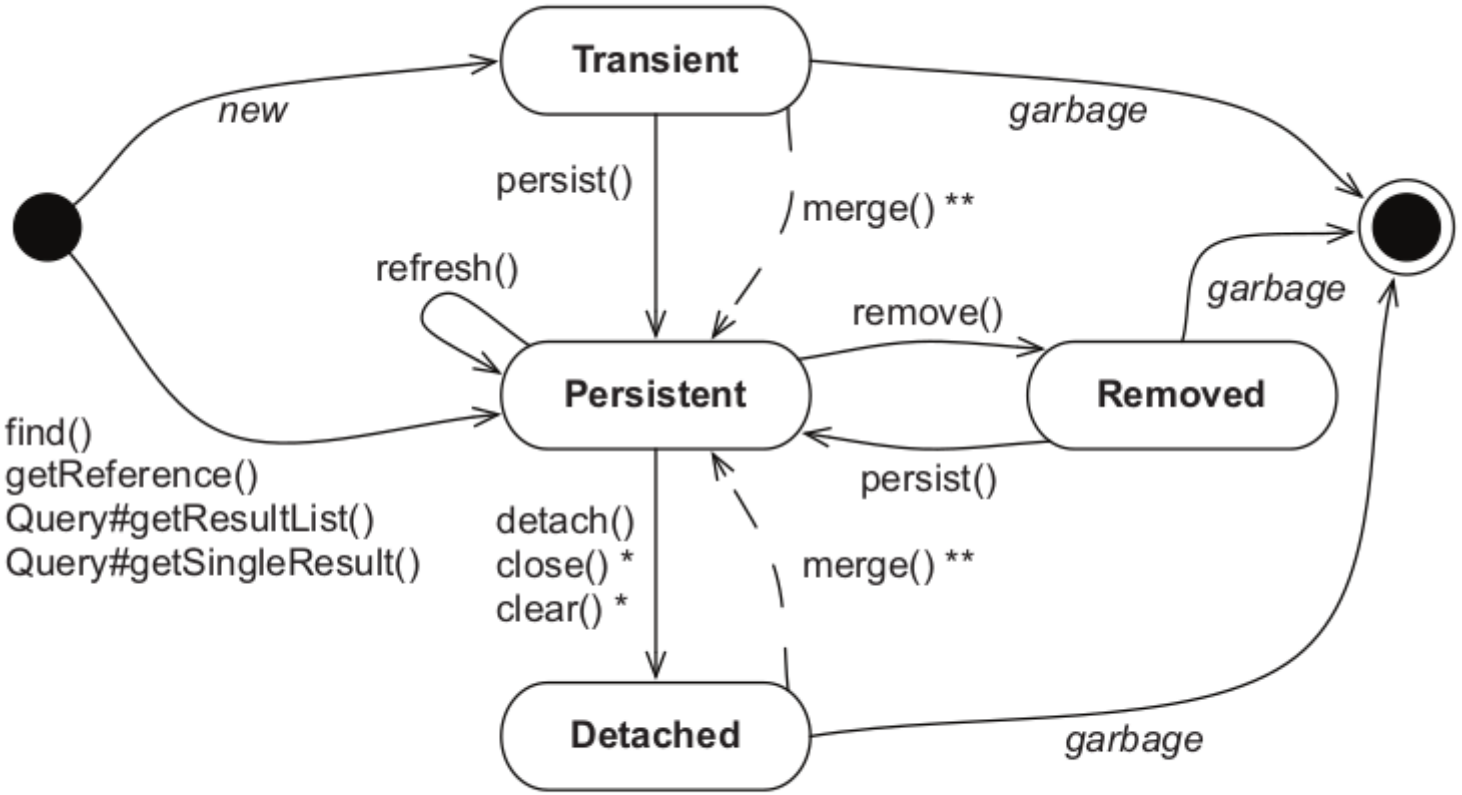
Persistent

- Objetos gestionados por el EntityManager.
- Asociados a un Contexto de Persistencia.
- Los cambios se guardan automáticamente (dirty checking)
- Se sincronizan con la BD en commits.

Puede pasar a:

- Removed → `remove()`
- Detached → `detach()`, `clear()`, o al cerrar el EntityManager.

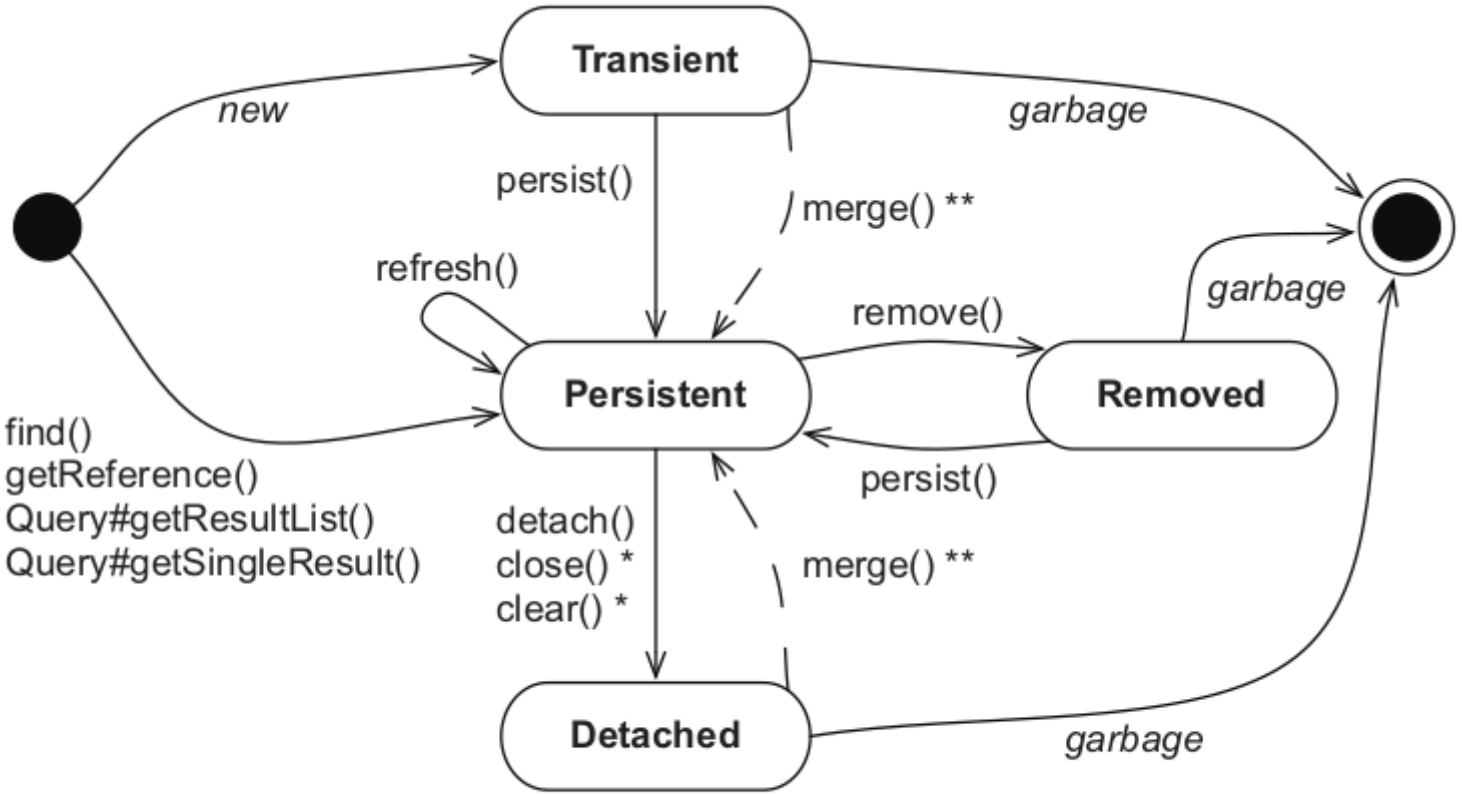
4. Persistencia con ORM – Estados



Removed

- Marcado para borrarse de la BD.
- Sigue gestionado hasta el commit.
- Se elimina definitivamente al confirmar la transacción

4. Persistencia con ORM – Estados



Detached

- Objeto que estuvo gestionado, pero ya no.
- Los cambios NO se guardan automáticamente.
- Puede volver a gestionarse con `merge()`

4. Persistencia con ORM – Lenguajes de consulta

JPQL
(Java Persistence Query Language)

- Sintaxis similar a SQL.
- No consultamos tablas, sino objetos.
  Devuelve objetos, no filas.
- Permite parámetros nombrados.
- Evita depender del esquema físico de la BBDD.
  Funciona igual, aunque cambie el SGBD.

```
String jpql = """
  SELECT l
  FROM Libro l
    JOIN l.categoria c
    WHERE c.nombre = :tipo
      AND LOWER(l.titulo) LIKE
        CONCAT('%', :palabra, '%')
    """;

TypedQuery<Libro> query = em.createQuery(
  jpql,
  Libro.class
);
query.setParameter("tipo", "Novela");
query.setParameter("palabra", "jurásico");
List<Libro> libros = query.getResultList();
```

SpringBoot y JPA

- SpringBoot ofrece una implementación automática de repositorios JPA
  - Interfaz anotadada `@Repository`
  - **`extends`** `JpaRepository<Entidad, TipoID>`

```
@Repository
public interface UsuarioRepositoryJpa
        extends JpaRepository<UsuarioEntity, String> {

}
```

Implementación automática

SpringBoot y JPA

- Consultas

```
@Repository
public interface UsuarioRepositoryJpa
        extends JpaRepository<UsuarioEntity, String> {

   @Query("SELECT u FROM UsuarioEntity WHERE nombre = ?1")
   UsuarioEntity findByNombre(String nombre);

}
```

SpringBoot y JPA

- Generación automática de consultas

```
@Repository
public interface UsuarioRepositoryJpa
        extends JpaRepository<UsuarioEntity, String> {

   UsuarioEntity findByNombre(String nombre);

}
```

SpringBoot y transaccionalidad

- Por defecto, cada método de JPA es transaccional
  - No hay atomicidad en los servicios
- Anotación @Transactional
  - Delimita transacciones
  - En un método: el método es transaccional
    - Por defecto (habitualmente): READ_COMMITTED

```
@Transactional
public Usuario incrementarPuntos(String usuarioId, int puntos) {
    ...
}
```

Sobre transacciones: https://planetscale.com/blog/database-transactions

Bibliografía

- Java Persistence with Spring Data and Hibernate. Catalin Tudose.